Documenta����o ��� Kurio NFT Marketplace

Documentação do Projeto — Kurio NFT Marketplace

Documento de apoio para explicar o projeto em entrevista. Cobre: o que foi feito, componentes (e onde são usados), APIs, mocks, autenticação e arquitetura.

---

1. Visão geral

Frontend de um marketplace de NFTs (desafio "Marketplace de NFTs"), implementado a partir do layout do Figma, **sem backend real**: toda a API (REST + WebSocket) é simulada no navegador com **MSW**.

O que o usuário consegue fazer

- Explorar a home (hero, promoções, catálogo com busca/filtros/ordenação/paginação, blog).
- Ver o detalhe de um NFT (edições, abas, relacionados, favoritar).
- Montar o carrinho (como visitante ou logado), aplicar cupom, alterar quantidades.
- Cadastrar-se / entrar / sair; sessão persistida e expiração tratada.
- Finalizar a compra: dados do coletor → carteira → revisão → pedido (pendente → confirmado ou recusado).
- Acompanhar o pedido (polling + tempo real via Socket.IO).
- Editar perfil (dados, senha, avatar) e carteiras.

Stack | Camada | Tecnologia | |---|---| | Build | Vite 8, TypeScript 5.9 (strict) | | UI | React 19, Tailwind v4 (tokens via `@theme`), componentes estilo shadcn/ui sobre Radix, lucide-react, sonner | | Rotas | TanStack Router (code-based, rotas lazy, `validateSearch`) | | Dados | TanStack Query, Axios | | Tempo real | socket.io-client | | Formulários | react-hook-form + zod 4 | | Dinheiro | decimal.js (valores ETH sempre como **string**) | | Mocks | MSW 2 + `@mswjs/socket.io-binding` | | Testes | Vitest (unit, 26), Playwright (E2E + visual), Lighthouse |

---

2. Estrutura de pastas

```
src/
  api/          contratos (zod), cliente Axios, endpoints, erros, chaves de cache,
session-bus
  app/          RootLayout, query-client, router
  components/   UI compartilhada (ui/*, layout/*, estados, NftArt, etc.)
  config/       env.ts (variáveis VITE_*)
  features/     account, cart, catalog, checkout, favorites, home, orders, session,
wallets
  lib/          money, format, utils, form-errors, out-of-scope, hooks utilitários
  mocks/        "backend" simulado: db, seed, domain, handlers, scenarios, socket,
panel
  realtime/     RealtimeProvider, EventApplier (apply.ts), status
  routes/       páginas (uma por rota)
e2e/            testes Playwright + baselines visuais
lighthouse/     script e relatórios
```

Organização por **feature** (hooks + componentes do domínio juntos) e **páginas finas** em `routes/` que compõem as features.

---

3. Rotas e páginas

Rota	Página	Acesso
`/`	`HomePage` (Hero, Promos, Catalog, Blog)	pública
`/nft/$nftId`	`NftDetailPage`	pública
`/carrinho`	`CartPage` (carrinho de visitante funciona)	pública
`/entrar`, `/cadastro`	`AuthPage`	pública
`/pagamento`	`CheckoutPage`	**privada**
`/pedido/$orderId`	`OrderPage`	**privada**
`/perfil`	`ProfilePage`	**privada**
`/carteiras`	`WalletsPage`	**privada**
`/mercado`	redireciona para `/#catalogo`	—
`*`	`NotFoundPage`	—

- Páginas pesadas são **lazy** (`CartPage`, `CheckoutPage`, `NftDetailPage`, `OrderPage`, `ProfilePage`, `WalletsPage`, `AuthPage`) com `Skeleton` como fallback.

- Filtros da home vivem na **URL** (`routes/home-search.ts`): `parseHomeSearch` é tolerante (valores inválidos viram padrão, nunca quebram), `toListParams` converte para a query da API.
- Rotas privadas usam `RequireAuth` ; o destino original vai em `?redirect=` e é validado por `safeRedirect` (evita open redirect).

---

4. Catálogo de componentes e onde são chamados

4.1 Layout (`components/layout`)

Componente	Responsabilidade	Usado em
`AppShell`	Estrutura da página: Header, `<main>` , Footer, TabBar mobile, aviso de realtime/sessão	`RootLayout`
`Header`	Navegação desktop, busca, contador do carrinho, menu do usuário (Dropdown), menu mobile (Sheet)	`AppShell`
`Footer`	Rodapé	`AppShell`
`TabBar`	Barra inferior mobile (home, carrinho, conta)	`AppShell`
`ScreenHeader`	Cabeçalho de tela mobile com voltar/título	`CartPage` , `CheckoutPage` , `NftDetailPage` , `AccountLayout`

Fluxos "bare" no mobile (checkout, etc.) escondem Header/TabBar.

4.2 Compartilhados (`components`)

Componente	Função	Usado em
`NftArt`	Imagem/arte do NFT com alt acessível	`NftCard` , `Hero` , `Promos` , `Blog` , `Filters` , `CartPage` , `CheckoutPage` , `NftDetailPage`
`states.tsx` → `ErrorState` , `EmptyState` , `InlineNotice`	Estados de erro/vazio/aviso padronizados	Catalog, Cart, Checkout, NftDetail, Order, Wallets, Profile, Auth, Blog, AppShell, RootLayout
`QuantityStepper`	Controle +/− de quantidade	`CartPage`
`Stars`	Avaliação	`NftDetailPage`
`form-field`	Label + Input + mensagem de erro ligados por `aria`	formulários de Auth, Checkout, Profile, Wallets
`live-announcer` → `LiveAnnouncer` , `announce()`	Região `aria-live` global; `announce` é chamado em carrinho, favoritos, sessão, checkout, catálogo	`RootLayout` (montagem); hooks (uso)

Componente	Função	Usado em
`icons`	Ícones SVG próprios	Header, TabBar, cards etc.

4.3 UI base (`components/ui` , estilo shadcn/Radix)

`Button` (quase todas as telas), `Input` , `Label` , `Checkbox` , `RadioGroup` (WalletPanel), `Select` (Catalog ordenação, WalletsPage), `Dialog` (AuthPage, NftDetailPage, OrderPage), `Sheet` (Header menu mobile, Catalog filtros mobile), `DropdownMenu` (Header), `Tabs` (NftDetailPage), `Slider` (Filters faixa de preço), `Skeleton` , `Spinner` , `Toaster/sonner` (RootLayout).

4.4 Features

home: `Hero` (usa `useFeatured`), `Promos` , `Blog` (usa `useEditorialPosts`) → todos em `HomePage` .

catalog

- `Catalog` (busca, filtros, ordenação, paginação, sheet de filtros mobile) ← `HomePage`
- `Filters` (categoria, faixa de preço, etc.) ← `Catalog`
- `NftCard` / `NftCardSkeleton` ← `Catalog` , `CartPage` (sugestões), `NftDetailPage` (relacionados)
- hooks: `useNftList` , `useFeatured` , `useNftDetail` , `useRelatedNfts` , `useEditorialPosts`

favorites: `useFavorites` , `useIsFavorite` , `useToggleFavorite` (otimista com rollback) ← `NftCard` , `NftDetailPage` . Requer login (visitante é levado ao login).

cart: hooks `useCart` , `useCartCount` (Header, TabBar), `useAddToCart` , `useUpdateCartItem` , `useRemoveCartItem` , `useApplyCoupon` , `useRemoveCoupon` e `useCartChanges` (detecta mudança de preço/disponibilidade e avisa) ← `CartPage` , `NftDetailPage` , `CheckoutPage` .

checkout

- `WalletPanel` (escolha de carteira, rede e provedor; conectar/desconectar simulado) ← `CheckoutPage`
- `QuoteDiff` → `diffQuotes` + `QuoteChanges` (banner "o total mudou", exige novo aceite) ← `CheckoutPage`
- `schema.ts` (zod do coletor, `isValidCpf` , `formatCpf`)
- `draft.ts` (rascunho do formulário por usuário em `sessionStorage` ; `clearAllCheckoutDrafts` no logout)
- `attempt.ts` (`attemptStore` : tentativa de pedido persistida com chave de idempotência)
- `useOrderSubmit` (máquina de envio com retries) e `useQuote` (cotação, `staleTime: 0`)

orders: `useOrder` (polling 3 s enquanto pendente), `usePendingOrders` ← `OrderPage` , `CheckoutPage` .

wallets: `useWallets` , `useSaveWallet` , `useWalletConnection` , `useConnectWallet` , `useDisconnectWallet` ← `WalletsPage` , `CheckoutPage` , `WalletPanel` .

account: `AccountLayout` (menu lateral/abas da conta) ← `ProfilePage`, `WalletsPage`; hooks `useUpdateProfile`, `useChangePassword`, `useUploadAvatar`, `useRemoveAvatar` ← `ProfilePage`.

session: `SessionProvider` / `useSession` (RootLayout; consumido por Header, TabBar, NftCard, hooks de carrinho/checkout...), `RequireAuth` ← Checkout, Order, Profile, Wallets; `storage.ts` (`authStorage`, `guestCart`).

4.5 Outros

- `realtime/RealtimeProvider` ← `RootLayout`; `EventApplier` (apply.ts) ← RealtimeProvider; `useRealtimeStatus` ← AppShell.
- `mocks/panel/MockPanel` (painel flutuante "Mocks", lazy) ← `RootLayout` quando mocks ligados.
- `lib/out-of-scope.ts` → `notAvailable()`: toast informativo para ações fora do escopo (links de rodapé, etc.).

---

5. APIs

5.1 Camada de cliente (`src/api`)

- **`contracts.ts`**: schemas zod + tipos TS de todas as respostas/requisições (fonte única de verdade do contrato). `EthString` é uma string decimal validada.
- **`endpoints.ts`**: objeto `api` agrupado (`api.auth`, `api.nfts`, `api.cart`, `api.orders`...) — componentes nunca chamam Axios direto.
- **`http.ts`**: instância Axios com:
  - timeout próprio via `AbortController`;
  - `Authorization: Bearer <token>` lido de `localStorage['kurio.auth']`;
  - cabeçalho `X-Guest-Cart` para o carrinho de visitante;
  - normalização de erros para `ApiError` (`errors.ts`: `status`, `code`, `fields`, `retryable`, `isSessionExpired`);
  - em **401**, emite no `sessionBus` (`session_expired` | `session_invalid`) — desacopla o HTTP do React.
- **`keys.ts`**: fábrica de query keys. Públicas: `['nfts', …]`, `['editorial']`. Privadas: `['user', userKey, …]` (userKey = id ou `'guest'`) — assim trocar de usuário nunca mostra dados do anterior.

5.2 Política do TanStack Query (`app/query-client.ts`)

- `staleTime` 30 s, `gcTime` 5 min; refetch ao focar/reconectar.
- Leituras: até 2 retries **só** se o erro for retryable, backoff `400·2ⁿ` (máx. 4 s). Mutations: sem retry.
- Mutations do carrinho **serializadas** (`scope: { id: 'cart' }`) para evitar corrida.
- Favoritos com atualização otimista + rollback.

5.3 Endpoints REST (simulados)

Área	Método e rota	Auth
Auth	`POST /api/auth/register`, `POST /api/auth/login`, `POST /api/auth/logout`, `GET /api/session`	logout/session exigem token
NFTs	`GET /api/nfts` (busca, filtros, ordenação, paginação), `GET /api/nfts/featured`, `GET /api/nfts/:id`, `GET /api/nfts/:id/related`	pública
Editorial	`GET /api/editorial`	pública
Favoritos	`GET/PUT/DELETE /api/favorites…`	privada
Carrinho	`GET /api/cart`, itens (POST/PATCH/DELETE), cupom (PUT/DELETE)	usuário **ou** visitante (`X-Guest-Cart`)
Cotação	`POST /api/quotes`	privada
Pedidos	`POST /api/orders` (idempotente, `Idempotency-Key`), `GET /api/orders/:id`, `GET /api/orders`	privada
Perfil	`PATCH /api/profile`, `PUT /api/profile/password`, avatar (PUT/DELETE)	privada
Carteiras	`GET/PUT /api/wallets…`, `/api/wallet-connection`	privada

Erros seguem um formato único `{ code, message, fields? }` (validação por campo vira erro no formulário via `lib/form-errors.ts → applyServerErrors`).

5.4 Tempo real (Socket.IO)

- Eventos: `nft.updated` (estoque/preço) e `order.updated` (status do pedido).
- Envelope: `{ eventId, type, resource, version, occurredAt, payload }`.
- `EventApplier` (cliente): **deduplica** por `eventId` (janela de 500), **descarta versões antigas**, confere dono do pedido e aplica o patch no cache do Query.
- Ao reconectar: `applier.reset()` + `invalidateQueries` → reconcilia com o REST (a verdade é sempre o servidor).
- Conexão só com transporte `websocket`, path `/socket.io/`, reconexão com backoff.

---

6. Mocks (`src/mocks`)

Por que MSW: intercepta no nível de rede, então o código do app é o **mesmo** que falaria com um backend real (Axios, headers, status HTTP, WebSocket de verdade).

Arquivo	Papel
`browser.ts`	`startMocks()`: `setupWorker`, instala `window.__KURIO_MOCK__`, retoma pedidos pendentes

Arquivo	Papel
`db.ts`	"Banco" em memória persistido em `localStorage` (`getDb`, `commit`, `resetDb`, geradores de id) com `SCHEMA_VERSION`
`seed.ts`	Dados iniciais: usuários demo, NFTs, editorial, carteiras, carrinho semeado. Senhas guardadas como **SHA-256 + salt** (`sha256.ts`, sem dependência)
`types.ts`	Registros internos (UserRecord, SessionRecord, OrderRecord…) separados dos contratos públicos
`domain/`	Regras de negócio puras: `auth`, `cart`, `catalog`, `quote`, `orders`, `events` (+ `domain.test.ts`)
`handlers/`	Um arquivo por área (auth, nfts, editorial, favorites, cart, orders, profile, wallets) montando as rotas MSW
`route.ts`	Helper `route(method, path, fn, {auth})`: aplica latência, falhas injetadas, autenticação e conversão de erros em resposta
`http-fail.ts`	`HttpFail` / `fail()` e erros de validação (zod → campos)
`scenarios.ts`	Cenários configuráveis (ver abaixo)
`faults.ts`	Latência, atrasos enfileirados e regras de falha por glob de rota
`socket.ts`	Servidor Socket.IO simulado (`ws.link` + `toSocketIo`): autenticação pelo evento `session.authenticate`, entrega por usuário
`control.ts`	API `window.__KURIO_MOCK__` (reset, trocar cenário, expirar sessão, resolver pedidos, mutar NFT…) usada pelos testes E2E e pelo painel
`panel/MockPanel.tsx`	UI flutuante "Mocks" para trocar cenário/forçar eventos durante a demo
`prng.ts`	PRNG determinístico (mulberry32) para latência/falhas reproduzíveis

Cenários (combináveis por `?scenario=a,b`): `default`, `fast`, `slow`, `jitter`, `empty`, `offline`, `server-error`, `flaky`, `forbidden`, `session-expired`, `session-short` (TTL 30 s), `signup-conflict`, `favorites-failure`, `price-change`, `sold-out`, `order-timeout`, `payment-declined`, `wallet-rejected`.

Dados de demonstração

- `nova@kurio.test` (carrinho semeado) e `rafael@kurio.test` — senha `Kurio#2026`.
- Cupons: `KURIO10`, `BEMVINDO5`, `EXPIRED20` (expirado, para testar erro).
- CPF válido para teste: `529.982.247-25`.

Pedido simulado: `createOrder` valida cotação, é idempotente (mesma `Idempotency-Key` → mesmo pedido), nasce `pending` e é resolvido após um tempo para `confirmed` ou `declined` (cenário). Ao confirmar, baixa estoque, remove itens comprados do carrinho e publica `nft.updated` + `order.updated`.

---

7. Autenticação (login e sessão)

7.1 Visão do fluxo

1. **Login/Cadastro** (`AuthPage`, react-hook-form + zod) chama `useSession().login/register`.
2. `SessionProvider` lê o `guestCartId` (`guestCart.peek()`) e envia junto no corpo: o servidor **mescla o carrinho de visitante** no do usuário (`mergeGuestIntoUser`: soma quantidades respeitando estoque e `maxPerOrder`, mantém cupom, apaga o carrinho visitante).
3. O servidor devolve `{ user, token, expiresAt }`. O front chama `establish()`: limpa o cache privado → grava `{ token, expiresAt, userId }` em `localStorage['kurio.auth']` → limpa o id de visitante → `status = authenticated`.
4. Chamadas seguintes: Axios injeta `Authorization: Bearer <token>`.
5. Após login o usuário vai para `redirect` (validado por `safeRedirect`) ou para a home.

7.2 Lado "servidor" (mock)

- `handlers/auth.ts`:
  - **register**: valida com `registerSchema`; erros `email_taken` / `username_taken` (409 com `fields`); cria usuário com salt + hash; cria sessão; faz o merge do carrinho.
  - **login**: confere `hashPassword(salt, senha)`; falha → `invalid_credentials` (mensagem genérica, não revela se o e-mail existe).
  - **logout**: remove a sessão do banco. **GET** `/api/session`: devolve o usuário e `expiresAt`.
- `domain/auth.ts`:
  - `createSession`: token opaco `kt_…`, TTL do cenário (padrão 8 h), poda sessões antigas.
  - `authenticate` → `AuthState` com `problem`: `missing | invalid | expired`.
  - `requireUser`: `expired` → 401 `session_expired`; demais → 401 `session_invalid`.
  - `ownerKeyOf`: `user:<id>` se logado, senão `guest:<X-Guest-Cart>` (é o que permite carrinho de visitante).

7.3 Estados da sessão (`SessionProvider`)

`loading` (há token, validando) · `guest` · `authenticated` · `error` (falha de rede ao validar; há `retry`).

- **Boot**: se existe token e não expirou, chama `GET /api/session` para validar/hidratar o usuário. Token vencido localmente é descartado sem chamar a API.
- **Expiração**: um `setTimeout` até `expiresAt` encerra a sessão proativamente.
- **401 em qualquer chamada** → `sessionBus` → `endSession`: limpa token, zera usuário, apaga cache privado, anuncia (a11y) e navega para `/entrar?redirect=<url atual>&motivo=expirada`. Assim o usuário volta exatamente de onde estava.

- **Logout**: tenta `POST /logout` (ignora falha), limpa token, rascunhos de checkout e tentativas, cache privado e desconecta o socket.
- **Multi-aba**: evento `storage` — logout em uma aba desloga as outras; login com outro usuário em outra aba recarrega a página (evita misturar dados).
- **Isolamento de dados**: toda query privada tem `userKey` na chave e o cache `['user']` é removido em login/logout/expiração → nenhum vazamento entre usuários (coberto por E2E).
- **Socket**: autentica com o token via evento `session.authenticate`; ao deslogar, desconecta.

7.4 Proteção de rotas

`RequireAuth` envolve Checkout, Pedido, Perfil e Carteiras: enquanto `loading` mostra skeleton; se visitante, redireciona (em `useEffect`, com refs para evitar loop) para `/entrar?redirect=…`. Ponto de atenção que foi corrigido: o redirect via `<Navigate>` com a URL "viva" causava loop infinito durante a transição de rota.

7.5 Pontos de segurança a mencionar (e limites honestos)

- Token em `localStorage` é simples e funciona para o escopo do desafio; em produção o ideal seria cookie `HttpOnly` + `SameSite` (mitiga XSS) e refresh token.
- `safeRedirect` impede open redirect; senha nunca é guardada em texto (mesmo no mock: hash + salt).
- O hash do mock é para realismo da demo, **não** substitui bcrypt/argon2 no servidor real.

---

8. Arquitetura e fluxo de dados

```
UI (routes/features) ──hooks──▶ TanStack Query ──▶ api/endpoints ──▶ Axios (http.ts)
                                       ▲                                    │
                              EventApplier│                                   ▼
                                       │                          MSW (Service
Worker)
              Socket.IO client ◀── WebSocket ── MSW socket.io-binding  │ handlers →
domain → db(localStorage)
```

Decisões-chave:

1. **Contratos em zod** compartilhados por front e mock → o mock não "mente" sobre o formato.
2. **Server state no TanStack Query**, estado de UI local; só sessão e realtime status são contextos/stores globais.
3. **Dinheiro como string decimal** (`decimal.js`) — nunca `number` em ETH (evita erro de ponto flutuante).

4. **Cache particionado por usuário** (`userKey`).
5. **Realtime como otimização, REST como verdade**: eventos atualizam o cache; reconexão sempre reconcilia com refetch.
6. **Resiliência**: retries seletivos, idempotência na criação de pedido, estados de erro/vazio em toda tela.
7. **Acessibilidade**: `aria-live` global, foco/roles em diálogos e status, lint `jsx-a11y` , sem overflow horizontal.
8. **Responsivo**: desktop ≥ 1024 px (`lg:`); mobile é a base, com fluxos "bare".
9. **Performance**: rotas lazy, chunks manuais só para React/TanStack; `socket.io-client` importado dinamicamente **depois** de `startMocks()` .

Armadilhas de integração que valem uma boa história

- `engine.io-client` captura `WebSocket` no carregamento do módulo; por isso o import dinâmico após iniciar o MSW (e sem compartilhar chunk com ele).
- MSW remove o prefixo `/socket.io/` ao casar — usar `ws.link(origin)` .
- O modo `offline` do mock precisa completar o handshake do Engine.IO antes de fechar, senão o cliente fica preso em "connecting".

---

9. Checkout (máquina de estados)

1. **Formulário** (coletor: nome, e-mail, CPF com máscara/validação, etc. + carteira). Rascunho salvo por usuário em `sessionStorage` .
2. **Revisão**: obtém cotação (`useQuote` , sempre fresca). Se `revision` /itens mudaram (preço, estoque, cupom) → fase **changes**: `QuoteChanges` mostra o diff e exige aceite explícito.
3. **Confirmar**: `useOrderSubmit` grava a tentativa (`attemptStore` com `Idempotency-Key`) e cria o pedido; em timeout/erro de rede tenta de novo até 3× (800/1600/3200 ms) **com a mesma chave** → nunca duplica pedido. Reabrir a página retoma a tentativa (`resume`).
4. **`/pedido/$orderId`** : `pending` (role=status, polling 3 s + realtime) → `confirmed` (Dialog com recibo, `tx-ref` , snapshot imutável) ou `declined` (carrinho preservado, botão de tentar de novo).
5. `/pagamento` com pedido pendente existente redireciona para ele (verificado uma vez via ref).
6. Ao sair de `pending` , invalida carrinho, pedidos e `['nfts']` (estoque mudou).

---

10. Testes e qualidade

- **Vitest (26)**: `money.test.ts` (aritmética ETH), `domain.test.ts` (regras de carrinho/cotação/pedido), `sha256.test.ts` .
- **Playwright** (`e2e/`): fluxos completos (login, carrinho, checkout confirmado/recusado, timeout idempotente, mudança de preço, expiração de sessão, reconexão realtime,

isolamento entre usuários, responsivo) + **15 screenshots de baseline** visuais. Usam `window.__KURIO_MOCK__` para determinismo.
- **Lighthouse** (`lighthouse/run.mjs` , resumo em `lighthouse/SUMMARY.md`): acessibilidade/boas práticas/SEO atendem; **performance mobile ficou ~75–78 (meta 90)** — causa: avaliação de JS sob throttling de CPU 4× incluindo o runtime do MSW; sem MSW ≈ 80. Justificado em `ARCHITECTURE.md` .
- `tsc` strict, ESLint (inclui `jsx-a11y`) e build limpos.

Comandos: `npm run dev` , `npm run build` , `npm run lint` , `npm test` , `npm run test:e2e` (ver `README.md`).

---

11. Desvios e limitações (seja transparente na entrevista)

- Cota da ferramenta do Figma acabou: **Pagamento desktop foi inferido do mobile**; itens de menu de Perfil/Carteiras inferidos pelos ícones.
- Ações sem tela no Figma/escopo mostram toast `notAvailable()` .
- Autenticação e pagamento são **simulados**; não há blockchain real (conexão de carteira é fake).
- Deploy (Vercel/Netlify) é feito pelo usuário; há `vercel.json` e `public/_redirects` para o fallback SPA, e `.env` mantém mocks ligados.

---

12. Perguntas prováveis e respostas curtas

Por que MSW em vez de json-server? Intercepta no nível de rede: o app usa Axios/WebSocket reais, o mesmo código serviria com backend de verdade, e dá para simular latência, falhas e Socket.IO.

Como evitou pedido duplicado? `Idempotency-Key` gerada uma vez por tentativa e persistida; retries reutilizam a chave e o servidor devolve o mesmo pedido.

Como lida com preço mudando no checkout? A revisão rebusca a cotação; se `revision` /issues divergem, entra na fase `changes` e o usuário precisa aceitar o novo total antes de criar o pedido.

E se a sessão expira no meio do fluxo? Qualquer 401 vai pelo `sessionBus` , a sessão é encerrada, o cache privado limpo e o usuário é levado a `/entrar?redirect=<onde estava>&motivo=expirada` ; após login volta ao ponto.

Como garante que um usuário não vê dados de outro? Query keys com `userKey` , remoção do cache `['user']` em login/logout/expiração, sincronização entre abas e eventos de socket filtrados por dono.

Por que strings para ETH? `number` perde precisão (0.1 + 0.2). `decimal.js` + string mantém exatidão e é o que uma API real enviaria.

Realtime e REST não conflitam? O `EventApplier` ignora eventos duplicados/antigos (por `eventId` e `version`); ao reconectar, tudo é revalidado via REST.

O que melhoraria com mais tempo? Cookies HttpOnly + refresh token, testes de acessibilidade automatizados (axe) no E2E, otimizar o bundle do MSW (só em dev/preview), virtualização de lista, i18n.

Onde está a regra de negócio dos mocks? `src/mocks/domain/*` (puras, testadas); os `handlers/*` só fazem parse/auth/serialização.